

R: limpieza, manipulación y exploración de datos

11mo encuentro: Introducción a Quarto

Jonatán Perren

9 de junio de 2026

Hasta ahora, el análisis de datos en R y la comunicación de sus resultados vivían en lugares separados: un gráfico terminaba copiado a una presentación, una tabla reformateada en Word. Si los datos cambiaban, había que repetir todo el proceso a mano.

Quarto resuelve esto integrando el análisis y el documento en un mismo archivo. El código R y el texto narrativo conviven en un `.qmd` y, al renderizarlo, Quarto ejecuta el código y genera el documento final. Si los datos cambian, basta con volver a renderizar.

La palabra clave es **reproducibilidad**: los resultados siempre son consistentes con el código y los datos que los generan, sin intervención manual.

¿Qué es Quarto?

Quarto es el sistema de publicación de documentos dinámicos de Posit. Es la evolución de R Markdown y soporta múltiples lenguajes (R, Python, Julia) y múltiples formatos de salida desde un mismo archivo.



Herramienta	Cómo funciona
R + Word/PowerPoint	el análisis vive en R y los resultados se copian manualmente al documento. Cualquier cambio en los datos implica repetir el proceso.
Quarto	texto y código conviven en el mismo <code>.qmd</code> . Al renderizar, el código se ejecuta y el documento final se genera actualizado.

Quarto viene incluido en las versiones recientes de RStudio. Desde la terminal: `quarto check` para verificar la instalación.

Proyectos de RStudio y estructura de carpetas

Ya conocen los proyectos de RStudio (`.Rproj`): fijan el directorio de trabajo y hacen que todas las rutas sean relativas a la raíz del proyecto. Quarto hereda esa misma lógica: el `.qmd` se renderiza con la raíz del proyecto como directorio de trabajo.

Hasta ahora, un proyecto típico podía verse así:

```
1 mi_proyecto
2 |-- mi_proyecto.Rproj
3 |-- datos
4 |   '-- encuesta.csv
5 |-- scripts
6   '-- limpieza.R
```

Ahora que también generamos reportes, el proyecto queda:

```
1 mi_proyecto
2 |-- mi_proyecto.Rproj
3 |-- datos
4 |   |-- brutos          <- datos originales, nunca se modifican
5 |   |   '-- encuesta.csv
6 |   '-- procesados      <- resultado de los scripts de limpieza
7 |       '-- encuesta_limpia.rds
8 |-- scripts
9 |   '-- limpieza.R      <- lee brutos, guarda en procesados
10 |-- reporte.qmd        <- lee procesados, no limpia datos
```

El `.qmd` no limpia datos: eso es trabajo de los scripts. El reporte carga directamente desde `datos/procesados/` y se dedica al análisis y la comunicación.

Crear un nuevo documento Quarto en RStudio

File → New File → Quarto Document...

Se abre un diálogo donde podés ingresar el título, el autor y elegir el formato de salida inicial (HTML, PDF o Word). Al confirmar, RStudio genera un `.qmd` con una plantilla mínima.

Elemento	Para qué sirve
Título y autor	completan el encabezado YAML automáticamente
Formato	define el <code>format</code> : inicial en el YAML (se puede cambiar después)
Plantilla generada	incluye un ejemplo mínimo de YAML, Markdown y un chunk

Guardá el archivo (**Ctrl+S**) dentro de la carpeta del proyecto antes de renderizar. El nombre del archivo se convierte en el nombre del documento generado.

Un archivo .qmd tiene tres componentes, siempre en este orden:

Componente	Descripción
Encabezado YAML	metadatos del documento: título, autor, formato de salida
Texto en Markdown	narrativa, explicaciones, interpretaciones
Chunks de código	bloques de código R que se ejecutan al renderizar

El YAML siempre va al principio. Markdown y chunks pueden alternarse libremente a lo largo del cuerpo del documento.

Un .qmd mínimo completo

```
1 ---
2 title: "Análisis de Gapminder"
3 author: "Jonatán Perren"
4 date: today
5 format: html
6 ---
7
8 ## Esperanza de vida en 2007
9
10 El siguiente gráfico muestra la distribución
11 de la esperanza de vida por continente.
12
13 ```{r}
14 #| include: false
15 library(tidyverse)
16 library(gapminder)
17 ```
18
19 ```{r}
20 gapminder |>
21   filter(year == 2007) |>
22   ggplot(aes(x = continent, y = lifeExp)) +
23     geom_boxplot()
24 ```
```

El encabezado YAML va al inicio del archivo, entre dos líneas de `---`. Define los metadatos y la configuración del documento.

```
1 ---
2 title: "Análisis de Gapminder"
3 subtitle: "FCE-UNL, 2025"
4 author: "Jonatán Perren"
5 date: today
6 lang: es
7 format: html
8 ---
```

`date: today` inserta automáticamente la fecha del renderizado. Para una fecha fija: `date: "2025-06-15"`.

El encabezado YAML: claves más frecuentes

Clave	Descripción
<code>title</code>	título del documento
<code>subtitle</code>	subtítulo (opcional)
<code>author</code>	autor o autores
<code>date</code>	fecha (<code>today</code> para la fecha actual de renderizado)
<code>lang</code>	idioma del documento: <code>es</code> , <code>en</code>
<code>format</code>	formato de salida: <code>html</code> , <code>pdf</code> , <code>docx</code>
<code>toc</code>	tabla de contenidos: <code>true</code> / <code>false</code>
<code>number-sections</code>	numerar secciones automáticamente: <code>true</code> / <code>false</code>

El cuerpo del documento se escribe en Markdown: una sintaxis minimalista que define estructura y énfasis.

```
1 # Sección principal
2
3 ## Subsección
4
5 Texto normal. Palabras en negrita e en cursiva.
6
7 - Elemento de lista no ordenada
8 - Otro elemento
9
10 1. Primer paso
11 2. Segundo paso
```

Sintaxis	Resultado
# Título	encabezado de nivel 1
## Sección	encabezado de nivel 2
texto	negrita
<i>*texto*</i>	<i>cursiva</i>
`código`	<code>código</code> monoespaciado inline
- elemento	ítem de lista no ordenada
1. elemento	ítem de lista ordenada
[texto](url)	hipervínculo

Los chunks de código

Un **chunk** es un bloque de código R ejecutable. Se delimita con tres backticks y la especificación del lenguaje.

```
1 ```{r}
2 mean(c(10, 20, 30))
3 ```
```

Al renderizar, Quarto ejecuta el código y embebe el resultado en el documento. El chunk de arriba produce:

```
1 [1] 20
```

Los chunks se pueden nombrar para referenciarlos: ````{r nombre-del-chunk}`. Los nombres no pueden contener espacios.

Opciones de chunk: la sintaxis #|

Cada chunk puede tener opciones que controlan qué aparece en el documento final. Se escriben dentro del chunk con el prefijo #| (hash-pipe).

```
1  ```{r}
2  #| echo: false
3  #| warning: false
4  #| fig-cap: "Distribución de la esperanza de vida (2007)"
5
6  gapminder |>
7    filter(year == 2007) |>
8    ggplot(aes(x = lifeExp)) +
9    geom_histogram(binwidth = 5)
10  ```
```

La sintaxis #| es la forma moderna recomendada por Quarto. El estilo anterior ```{r echo=FALSE} sigue funcionando pero se desaconseja.

Opciones de chunk más frecuentes

Opción	Tipo	Default	Efecto
<code>echo</code>	lógico	true	¿Mostrar el código en el documento?
<code>eval</code>	lógico	true	¿Ejecutar el código?
<code>warning</code>	lógico	true	¿Mostrar advertencias de R?
<code>message</code>	lógico	true	¿Mostrar mensajes de R?
<code>include</code>	lógico	true	¿Incluir código y output?
<code>fig-width</code>	número	7	Ancho de la figura (pulgadas)
<code>fig-height</code>	número	5	Alto de la figura (pulgadas)
<code>fig-cap</code>	texto	—	Leyenda de la figura

echo: false — ocultar el código, mostrar el resultado

```
1 ```{r}
2 #| echo: false
3 mean(c(10, 20, 30))
4 ```
```

El documento muestra [1] 20 sin el código que lo generó.

eval: false — mostrar el código, no ejecutarlo

```
1 ```{r}
2 #| eval: false
3 install.packages("gapminder")
4 ```
```

Útil para mostrar instrucciones que el lector debe ejecutar por su cuenta.

include: false — ejecutar en silencio

```
1 ```{r}
2 #| include: false
3 library(tidyverse)
4 library(gapminder)
5 ```
```

El chunk carga los paquetes pero no aparece nada en el documento.

Además de los chunks, se puede insertar código R **dentro del texto** con la sintaxis ``r expr``. El resultado reemplaza la expresión directamente en la prosa.

```
1 El dataset contiene `r nrow(gapminder)` observaciones  
2 de `r n_distinct(gapminder$country)` países.
```

Produce: El dataset contiene **1704** observaciones de **142** países.

El código inline se actualiza con cada renderización. Si los datos cambian, los números en el texto cambian solos. Es la herramienta clave para evitar inconsistencias entre el análisis y la narrativa.

Renderizar el documento

Renderizar es el proceso que ejecuta todo el código R y combina los resultados con el texto Markdown para generar el documento final.

Desde RStudio

Hacé clic en el botón **Render** (atajo: **Ctrl + Shift + K**).

Desde la consola de R

```
1 quarto::quarto_render("mi_reporte.qmd")
```

El renderizado ejecuta el código en una sesión limpia desde cero. Si un objeto fue creado en la consola pero no está en el `.qmd`, el render fallará. El documento debe ser completamente autosuficiente.

El campo `format` del YAML determina el formato del documento generado.

Un solo formato

```
1 format: html
```

Múltiples formatos desde el mismo .qmd

```
1 format:  
2   html:  
3     toc: true  
4   pdf:  
5     number-sections: true  
6   docx: default
```

Para generar PDF, Quarto requiere una instalación de $\text{\LaTeX}$. La forma más simple es `tinytex::install_tinytex()`.

Formatos de salida: ¿cuándo usar cada uno?

Formato	Ventajas	Desventajas	Cuándo usarlo
HTML	interactivo, sin dependencias	necesita navegador	exploración, reportes web
PDF	tipografía profesional	requiere $\text{\LaTeX}$	documentos formales, TPI
Word	editable, familiar	difícil de personalizar	entregas institucionales

1. Creá un nuevo archivo `.qmd` en RStudio (File > New File > Quarto Document). Poné un título y tu nombre como autor. Guardalo y hacé **Render** a HTML. ¿Qué genera el archivo por defecto?
2. Reemplazá el contenido con un chunk `#| include: false` que cargue `tidyverse` y `gapminder`, y otro que muestre `head(gapminder)`. ¿Qué aparece en el documento?
3. Agregá un párrafo de texto antes del chunk con `head(gapminder)` que describa brevemente el dataset. Usá al menos un encabezado de nivel 2, una palabra en negrita y una en cursiva.
4. Cambiá el formato de salida a `pdf` y volvé a renderizar. Si no tenés $\text{\LaTeX}$ instalado, instalalo con `tinytex::install_tinytex()` y reiniciá RStudio.
5. Creá un chunk con `#| echo: false` que grafique la distribución de `LifeExp` con `geom_histogram()`. ¿Qué se ve en el documento final? ¿Qué no se ve?
6. Agregá una oración con código inline que reporte cuántas filas tiene `gapminder` usando ``r nrow(gapminder)``.

Creá tu primer reporte reproducible con `gapminder` (del paquete `gapminder`). Entregá el archivo `.qmd` y el HTML generado.

1. Creá un `.qmd` con título, autor, fecha automática y formato HTML con tabla de contenidos.
2. Organizá el reporte en tres secciones: «Descripción del dataset», «Distribución de la esperanza de vida» y «Comparación entre continentes».
3. En la primera sección, describí el dataset en texto Markdown e incluí una oración con código inline que reporte cuántos países y cuántos años contiene.
4. En la segunda sección, incluí un gráfico de distribución de `lifeExp` en `2007` con el código oculto y un título descriptivo con `labs()`.
5. En la tercera sección, mostrá el código y el resultado de una comparación de la esperanza de vida promedio por continente.